Student Management API Models

This section defines the main entity and data transfer objects used by the Student Management API. The models separate database entities from request payloads, making the API easier to organize, validate, and maintain.

Class Entity

The **Class** entity represents a class record in the database. It includes a unique identifier, an optional class name, and a collection of related students.

```
namespace StudentManagementAPI.Models.Entities
{
    public class Class
    {
        public Guid ClassId { get; set; }
        public string? ClassName { get; set; }
        public ICollection<Student>? Students { get; set; }
    }
}
```

Student Entity

The **Student** entity represents a student record in the database. It stores the student's name and email address, and uses **ClassId** to link each student to a related class.

```
namespace StudentManagementAPI.Models.Entities
{
    public class Student
    {
        public Guid StudentId { get; set; }
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
        public Guid ClassId { get; set; }
        public Class? Class { get; set; }
    }
}
```

Add DTOs

Add DTOs define the data required when creating new records through the API.

```
namespace StudentManagementAPI.Models
{
    public class AddClassDto
    {
        public string? ClassName { get; set; }
    }
}
```

```
namespace StudentManagementAPI.Models
{
    public class AddStudentDto
    {
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
        public Guid ClassId { get; set; }
    }
}
```

Update DTOs

Update DTOs define the fields that can be changed when modifying existing records. In this selection, the update model is used for updating class information.

```
namespace StudentManagementAPI.Models
{
    public class UpdateClassDto
    {
        public string? ClassName { get; set; }
    }
}
```

Classes API Controller

The **ClassesController** manages class records in the Student Management API. It provides endpoints to retrieve all classes, find a class by ID, add a new class, update class details, and delete a class from the database.

```
using Microsoft.AspNetCore.Mvc;
using StudentManagementAPI.Data;
using StudentManagementAPI.Models;
using StudentManagementAPI.Models.Entities;
```

```
namespace StudentManagementAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ClassesController : ControllerBase
    {
        private readonly ApplicationDbContext _dbContext;

        public ClassesController(ApplicationDbContext dbContext)
        {
            _dbContext = dbContext;
        }

        [HttpGet]
        public IActionResult GetAllClasses()
        {
            var classes = _dbContext.Classes.ToList();
            return Ok(classes);
        }

        [HttpGet("{id:guid}")]
        public IActionResult GetClassById(Guid id)
        {
            var classEntity = _dbContext.Classes.Find(id);

            if (classEntity is null)
            {
                return NotFound();
            }

            return Ok(classEntity);
        }

        [HttpDelete("{id:guid}")]
        public IActionResult DeleteClassById(Guid id)
        {
            var classEntity = _dbContext.Classes.Find(id);

            if (classEntity is null)
```

```
            {
                return NotFound();
            }

            _dbContext.Classes.Remove(classEntity);
            _dbContext.SaveChanges();

            return Ok(classEntity);
        }

        [HttpPost]
        public IActionResult AddClass(AddClassDto addClassDto)
        {
            var classEntity = new Class
            {
                ClassName = addClassDto.ClassName
            };

            _dbContext.Classes.Add(classEntity);
            _dbContext.SaveChanges();

            return Ok(classEntity);
        }

        [HttpPut("{id:guid}")]
        public IActionResult UpdateClass(Guid id, UpdateClassDto
updateClassDto)
        {
            var classEntity = _dbContext.Classes.Find(id);

            if (classEntity is null)
            {
                return NotFound();
            }

            classEntity.ClassName = updateClassDto.ClassName;
            _dbContext.SaveChanges();

            return Ok(classEntity);
```

```
        }
    }
}
```

Students API Controller

The **StudentsController** manages student records in the Student Management API. It provides endpoints to retrieve all students, find a student by ID, add a new student, update student details, and delete a student from the database.

```
using Microsoft.AspNetCore.Mvc;
using StudentManagementAPI.Data;
using StudentManagementAPI.Models;
using StudentManagementAPI.Models.Entities;

namespace StudentManagementAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        private readonly ApplicationDbContext _dbContext;

        public StudentsController(ApplicationDbContext dbContext)
        {
            _dbContext = dbContext;
        }

        [HttpGet]
        public IActionResult GetAllStudents()
        {
            var students = _dbContext.Students.ToList();
            return Ok(students);
        }

        [HttpGet("{id:guid}")]
        public IActionResult GetStudentById(Guid id)
        {
            var studentEntity = _dbContext.Students.Find(id);
```

```
        if (studentEntity is null)
        {
            return NotFound();
        }

        return Ok(studentEntity);
    }

    [HttpDelete("{id:guid}")]
    public IActionResult DeleteStudentById(Guid id)
    {
        var studentEntity = _dbContext.Students.Find(id);

        if (studentEntity is null)
        {
            return NotFound();
        }

        _dbContext.Students.Remove(studentEntity);
        _dbContext.SaveChanges();

        return Ok(studentEntity);
    }

    [HttpPost]
    public IActionResult AddStudent(AddStudentDto addStudentDto)
    {
        var studentEntity = new Student
        {
            FirstName = addStudentDto.FirstName,
            LastName = addStudentDto.LastName,
            Email = addStudentDto.Email,
            ClassId = addStudentDto.ClassId
        };

        _dbContext.Students.Add(studentEntity);
        _dbContext.SaveChanges();
```

```
            return Ok(studentEntity);
        }

        [HttpPut("{id:guid}")]
        public IActionResult UpdateStudent(Guid id, UpdateStudentDto
updateStudentDto)
        {
            var studentEntity = _dbContext.Students.Find(id);

            if (studentEntity is null)
            {
                return NotFound();
            }

            studentEntity.FirstName = updateStudentDto.FirstName;
            studentEntity.LastName = updateStudentDto.LastName;
            studentEntity.Email = updateStudentDto.Email;
            _dbContext.SaveChanges();

            return Ok(studentEntity);
        }
    }
}
```

SQL Stored Procedures

This section defines stored procedures for joining class and student data, retrieving student details, updating student records, deleting student records, and adding new students.

Class and Student Join Procedures

These procedures return combined data from the **Students** and **Classes** tables using different join types: inner join, left join, and right join.

```
CREATE PROCEDURE SP_ClassStudentInnerJoin
AS
BEGIN
    SELECT *
    FROM Students AS s
```

```
    INNER JOIN Classes AS c
        ON s.ClassId = c.ClassId;
END;

EXEC SP_ClassStudentInnerJoin;
```

```
CREATE PROCEDURE SP_ClassStudentLeftJoin
AS
BEGIN
    SELECT *
    FROM Students AS s
    LEFT JOIN Classes AS c
        ON s.ClassId = c.ClassId;
END;

EXEC SP_ClassStudentLeftJoin;
```

```
CREATE OR ALTER PROCEDURE SP_ClassStudentRightJoin
AS
BEGIN
    SELECT *
    FROM Students AS s
    RIGHT JOIN Classes AS c
        ON s.ClassId = c.ClassId;
END;
GO

EXEC SP_ClassStudentRightJoin;
```

Student Detail Procedures

These procedures manage individual student records by ID. They support retrieving, updating, deleting, and inserting student data.

```
CREATE PROCEDURE SP_GetStudentDetailsById
(
    @StudentId UNIQUEIDENTIFIER
)
AS
```

```
BEGIN
    SELECT *
    FROM Students
    WHERE StudentId = @StudentId;
END;

EXEC SP_GetStudentDetailsById
    @StudentId = "1E631E6E-B8A7-47DF-A14A-10821792E7DB";

CREATE PROCEDURE SP_UpdateStudentDetailsById
(
    @StudentId UNIQUEIDENTIFIER,
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50),
    @Email VARCHAR(50)
)
AS
BEGIN
    UPDATE Students
    SET FirstName = @FirstName,
        LastName = @LastName,
        Email = @Email
    WHERE StudentId = @StudentId;
END;

EXEC SP_UpdateStudentDetailsById
    @StudentId = "1E631E6E-B8A7-47DF-A14A-10821792E7DB",
    @FirstName = "Raju",
    @LastName = "Weds",
    @Email = "Rambhai";

CREATE PROCEDURE SP_DeleteStudentDetailsById
(
    @StudentId UNIQUEIDENTIFIER
)
AS
BEGIN
    DELETE FROM Students
    WHERE StudentId = @StudentId;
END;
```

```
EXEC SP_DeleteStudentDetailsById
    @StudentId = "1E631E6E-B8A7-47DF-A14A-10821792E7DB";

ALTER PROCEDURE SP_AddStudents
(
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50),
    @Email VARCHAR(50),
    @ClassId UNIQUEIDENTIFIER
)
AS
BEGIN
    INSERT INTO Students
    (
        StudentId,
        FirstName,
        LastName,
        Email,
        ClassId
    )
    VALUES
    (
        NEWID(),
        @FirstName,
        @LastName,
        @Email,
        @ClassId
    );
END;

EXEC SP_AddStudents
    @FirstName = "Rajus",
    @LastName = "Wedis",
    @Email = "Rahhmbhai",
    @ClassId = "F90FAC29-61A7-423C-9D12-E3F0EADE08EC";
```

Authentication and Authorization Setup

To add authentication and authorization support, install the required ASP.NET Core Identity and Entity Framework Core packages:

- **Microsoft.AspNetCore.Identity.EntityFrameworkCore** — provides ASP.NET Core Identity support with Entity Framework Core.
- **Microsoft.EntityFrameworkCore.InMemory** — provides an in-memory database provider for testing or lightweight development scenarios.

Authentication Database Context

The **AuthDbContext** class extends **IdentityDbContext<IdentityUser>**, which enables the API to store and manage user identity data through Entity Framework Core.

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace StudentManagementAPI.Data
{
    public class AuthDbContext : IdentityDbContext<IdentityUser>
    {
        public AuthDbContext(DbContextOptions<AuthDbContext> options)
: base(options)
        {
        }
    }
}
```

Application Startup Configuration

This section configures the application services and HTTP request pipeline for the Student Management API. It sets up authentication, authorization, CORS, Swagger documentation, database contexts, and controller routing.

Configured Services

- **Authentication database:** Uses an in-memory database named **AuthDb** for identity-related data.
- **Authorization:** Enables authorization services for secured endpoints.
- **CORS:** Adds an **AllowDev** policy that allows any origin, method, and header during development.
- **Identity endpoints:** Registers ASP.NET Core Identity API endpoints using **IdentityUser** and **AuthDbContext**.
- **Swagger:** Adds OpenAPI documentation with Bearer token support.
- **Application database:** Connects **ApplicationDbContext** to the SQL Server connection string named **DefaultConnection**.

Program.cs Code

```
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using StudentManagementAPI.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddDbContext<AuthDbContext>(options =>
    options.UseInMemoryDatabase("AuthDb"));

builder.Services.AddAuthorization();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowDev", policy =>
        policy.AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader());
});

builder.Services.AddIdentityApiEndpoints<IdentityUser>()
```

```
    .AddEntityFrameworkStores<AuthDbContext>();

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new Microsoft.OpenApi.Models.OpenApiInfo
    {
        Title = "Student Management API",
        Version = "v1"
    });

    options.AddSecurityDefinition("Bearer", new
Microsoft.OpenApi.Models.OpenApiSecurityScheme
    {
        In = Microsoft.OpenApi.Models.ParameterLocation.Header,
        Description = "Please enter a token",
        Name = "Authorization",
        Type = Microsoft.OpenApi.Models.SecuritySchemeType.Http,
        BearerFormat = "JWT",
        Scheme = "Bearer"
    });

    options.AddSecurityRequirement(new
Microsoft.OpenApi.Models.OpenApiSecurityRequirement
    {
        {
            new Microsoft.OpenApi.Models.OpenApiSecurityScheme
            {
                Reference = new
Microsoft.OpenApi.Models.OpenApiReference
                {
                    Type =
Microsoft.OpenApi.Models.ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            Array.Empty<string>()
        }
```

```
    });
});

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("De
faultConnection")));

var app = builder.Build();

app.UseCors("AllowDev");
app.MapIdentityApi<IdentityUser>();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```